



Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN  
BUSINESS ANALYTICSCarnegie Mellon University  
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

# Building a Convolutional Neural Network in PyTorch

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch** 4

Share

Share

Post

Neural networks are built with layers connected to each other. There are many different kind of layers. For image related applications, you can always find convolutional layers. It is a layer with very few parameters but applied over a large sized input. It is powerful because it can preserve the spatial structure of the image. Therefore it is used to produce state-of-the-art results on computer vision neural networks. In this post, you will learn about the convolutional layer and the network it built. After completing this post, you will know:

- What are convolutional layers and pooling layers
- How they fit together in a neural network
- How a neural network using convolutional layers is designed

**Kick-start your project** with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Building a Convolutional Neural Network in PyTorch  
Photo by Donna Elliot. Some rights reserved.

## Overview

This post is divided into four parts; they are

- The Case for Convolutional Neural Networks

- Building Blocks of Convolutional Neural Networks
- An Example of Convolutional Neural Network
- What Are in Feature Maps?

## The Case for Convolutional Neural Networks

Let's consider to make a neural network to process grayscale image as input, which is the simplest use case in deep learning for computer vision.

A grayscale image is an array of pixels. Each pixel is usually a value in a range of 0 to 255. An image with size 32×32 would have 1024 pixels. Taking it as input to a neural network would mean the first layer to have at least 1024 input weights.

Looking at the pixel value has little use to understanding the picture, because data is hiding in the spatial structure (e.g., whether there is a horizontal line or a vertical line on the picture). Hence the traditional neural network will find it difficult to find out information from image input.

Convolutional neural network is to use convolutional layers to preserve spatial information of pixels. It learns how alike are the neighboring pixels and generating **feature representations**. What the convolutional layers see from the picture is invariant to distortion in some degree. For example, a convolutional neural network could predict the same result even if the input image has shift in color, rotated or rescaled. Moreover, convolutional layers has fewer weights, thus easier to train.

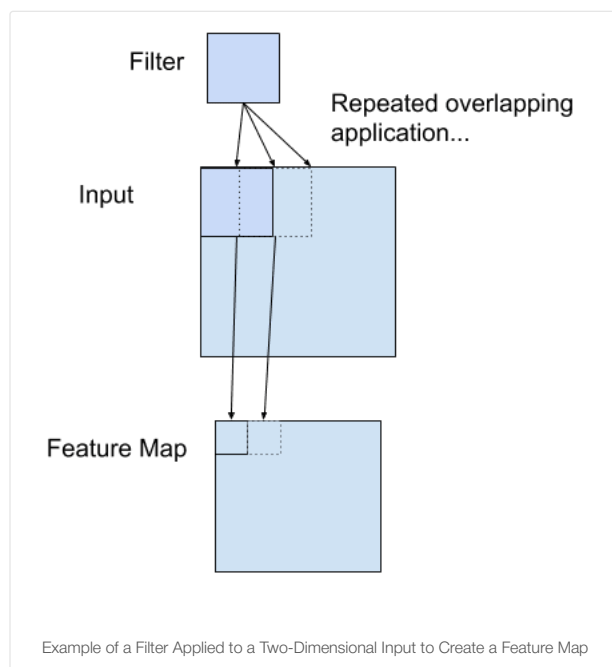
## Building Blocks of Convolutional Neural Networks

The simplest use case of a convolutional neural network is for classification. You will find it to contain three types of layers:

1. Convolutional layers
2. Pooling layers
3. Fully-connected layers

Neurons on a convolutional layer is called the filter. Usually it is a 2D convolutional layer in image application. The filter is a 2D patch (e.g., 3×3 pixels) that is applied on the input image pixels. The size of this 2D patch is also called the receptive field, meaning how large a portion of the image it can see at a time.

The filter of a convolutional layer is to multiply with the input pixels, and then sum up the result. This result is one pixel value at the output. The filter will move around the input image to fill out all pixel values at the output. Usually multiple filters are applied to the same input, producing multiple output tensors. These output tensors are called the **feature maps** produced by this layer. They are stacked together as one tensor and pass on to the next layer as input.



The output of a convolutional layer is called feature maps because usually it learned about the features of the input image. For example, whether there are vertical lines at the position. Learning the features from pixels is to help understanding the image at a higher level. Multiple convolutional layers are stacked together in order to infer higher level features from lower level details.

Pooling layer is to **downsample** the previous layer's feature map. It is usually used after a convolutional layer to consolidate features learned. It can compress and generalize the feature representations. A pooling layer also has a receptive field and usually it is to take the average (average pooling) or the

maximum (max pooling) over all values on the receptive field.

Fully connected layers are usually the final layers in a network. It is to take the features consolidated by previous convolutional and pooling layers as input to produce prediction. There might be multiple fully connected layers stacked together. In the case of classification, you usually see the output of the final fully connected layer applied with a softmax function to produce probability-like classification.

## Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

## An Example of Convolutional Neural Network

The following is a program to do image classification on the CIFAR-10 dataset.

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision
5
6 transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])
7
8 trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
9 testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
10
11 batch_size = 32
12 trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
13 testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=True)
14
15 class CIFAR10Model(nn.Module):
16     def __init__(self):
17         super().__init__()
18         self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
19         self.act1 = nn.ReLU()
20         self.drop1 = nn.Dropout(0.3)
21
22         self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
23         self.act2 = nn.ReLU()
24         self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))
25
26         self.flat = nn.Flatten()
27
28         self.fc3 = nn.Linear(8192, 512)
29         self.act3 = nn.ReLU()
30         self.drop3 = nn.Dropout(0.5)
31
32         self.fc4 = nn.Linear(512, 10)
33
34     def forward(self, x):
35         # input 3x32x32, output 32x32x32
36         x = self.act1(self.conv1(x))
37         x = self.drop1(x)
38         # input 32x32x32, output 32x32x32
39         x = self.act2(self.conv2(x))
40         # input 32x32x32, output 32x16x16
41         x = self.pool2(x)
42         # input 32x16x16, output 8192
43         x = self.flat(x)
44         # input 8192, output 512
45         x = self.act3(self.fc3(x))
46         x = self.drop3(x)
47         # input 512, output 10
48         x = self.fc4(x)
49         return x
50
51 model = CIFAR10Model()
52 loss_fn = nn.CrossEntropyLoss()
53 optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
54
55 n_epochs = 20
56 for epoch in range(n_epochs):
57     for inputs, labels in trainloader:
58         # forward, backward, and then weight update
59         y_pred = model(inputs)
60         loss = loss_fn(y_pred, labels)
61         optimizer.zero_grad()
62         loss.backward()
63         optimizer.step()

```

```

64
65     acc = 0
66     count = 0
67     for inputs, labels in testloader:
68         y_pred = model(inputs)
69         acc += (torch.argmax(y_pred, 1) == labels).float().sum()
70         count += len(labels)
71     acc /= count
72     print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
73
74 torch.save(model.state_dict(), "cifar10model.pth")

```

The CIFAR-10 dataset provides images in 32x32 pixels in RGB color (i.e., 3 color channels). There are 10 classes, labelled in integers 0 to 9. Whenever you are working on PyTorch neural network models for images, you will find the sister library torchvision useful. In the above, you used it to download the CIFAR-10 dataset from the Internet and transform it into a PyTorch tensor:

```

1 ...
2 transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])
3
4 trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
5 testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

```

You also used a DataLoader in PyTorch to help creating batches for training. Training is to optimize the cross entropy loss of the model, using stochastic gradient descent. It is a classification model, hence accuracy of classification is more intuitive than cross entropy, which is computed at the end of each epoch, by comparing the maximum value in the output logit to the dataset's labels:

```

1 ...
2 acc += (torch.argmax(y_pred, 1) == labels).float().sum()

```

It takes time to run the program above to train the network. This network should be able to achieve above 70% accuracy in classification.

It is typical in a network for image classification to be comprised of convolutional layers at an early stage, with dropout and pooling layers interleaved. Then, at a later stage, the output from convolutional layers is flattened and processed by some fully connected layers.

## What Are in Feature Maps?

There are two convolutional layers in the network defined above. They are both defined with kernel size of 3x3, hence it is looking at 9 pixels at a time to produce one output pixel. Note that the first convolutional layer is taking the RGB image as input. Hence each pixel has three channels. The second convolutional layer is taking a feature map with 32 channels as input. Each "pixel" as it sees will have 32 values. Thus the second convolutional layer has more parameters even they have the same receptive field.

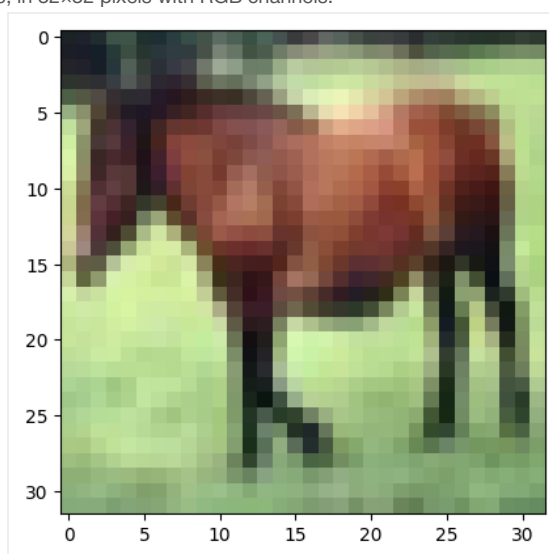
Let's see what is in the feature map. Let's say we pick one input sample from the training set:

```

1 import matplotlib.pyplot as plt
2
3 plt.imshow(trainset.data[7])
4 plt.show()

```

You should see that this is an image of a horse, in 32x32 pixels with RGB channels:



First, you need to convert this into a PyTorch tensor and make it a batch of one image. PyTorch models expect each image as a tensor in the format of (channel, height, width) but the data you read is in the format of (height, width, channel). If you use torchvision to transform the image into PyTorch tensors, this format conversion is done automatically. Otherwise, you need to **permute** the dimensions before use.

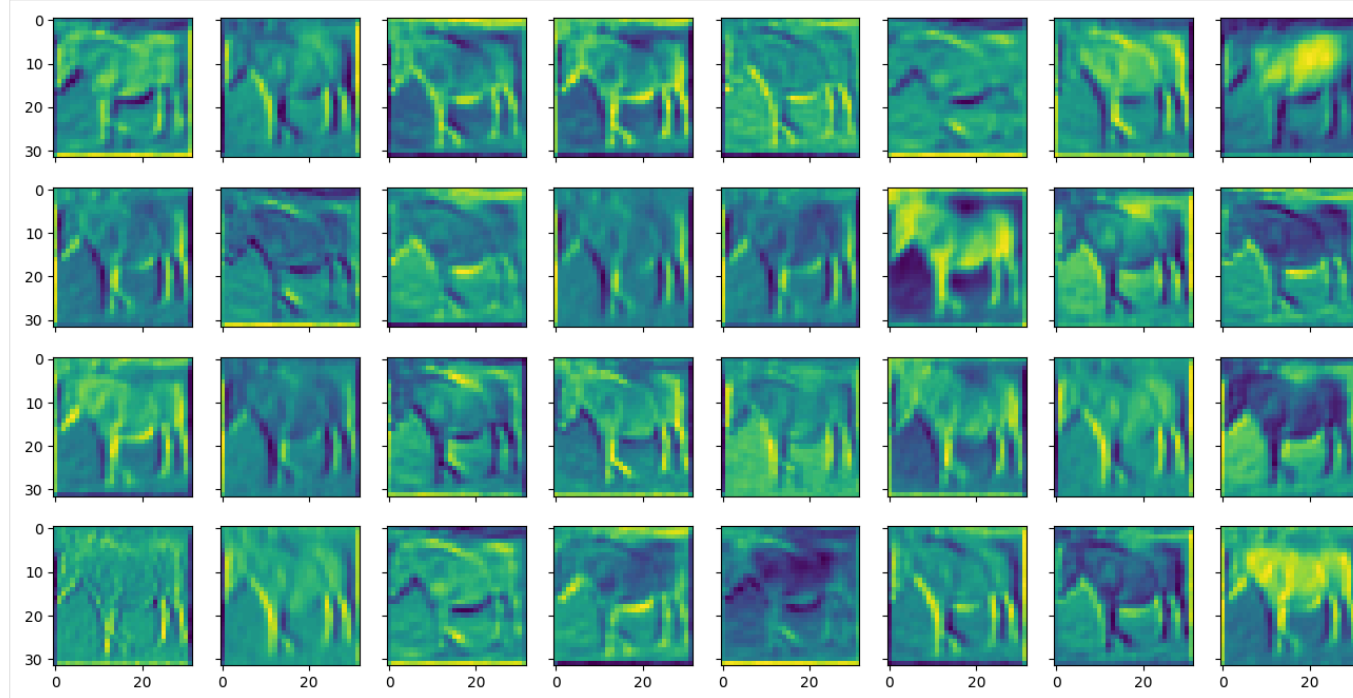
Afterward, pass it on through the model's first convolution layer and capture the output. You need to tell PyTorch that no gradient is needed for this calculation as you are not going to optimize the model weight:

```
1 X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
2 model.eval()
3 with torch.no_grad():
4     feature_maps = model.conv1(X)
```

The feature maps are in one tensor. You can visualize them using matplotlib:

```
1 fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
2 for i in range(0, 32):
3     row, col = i//8, i%8
4     ax[row][col].imshow(feature_maps[0][i])
5 plt.show()
```

You may see the following:

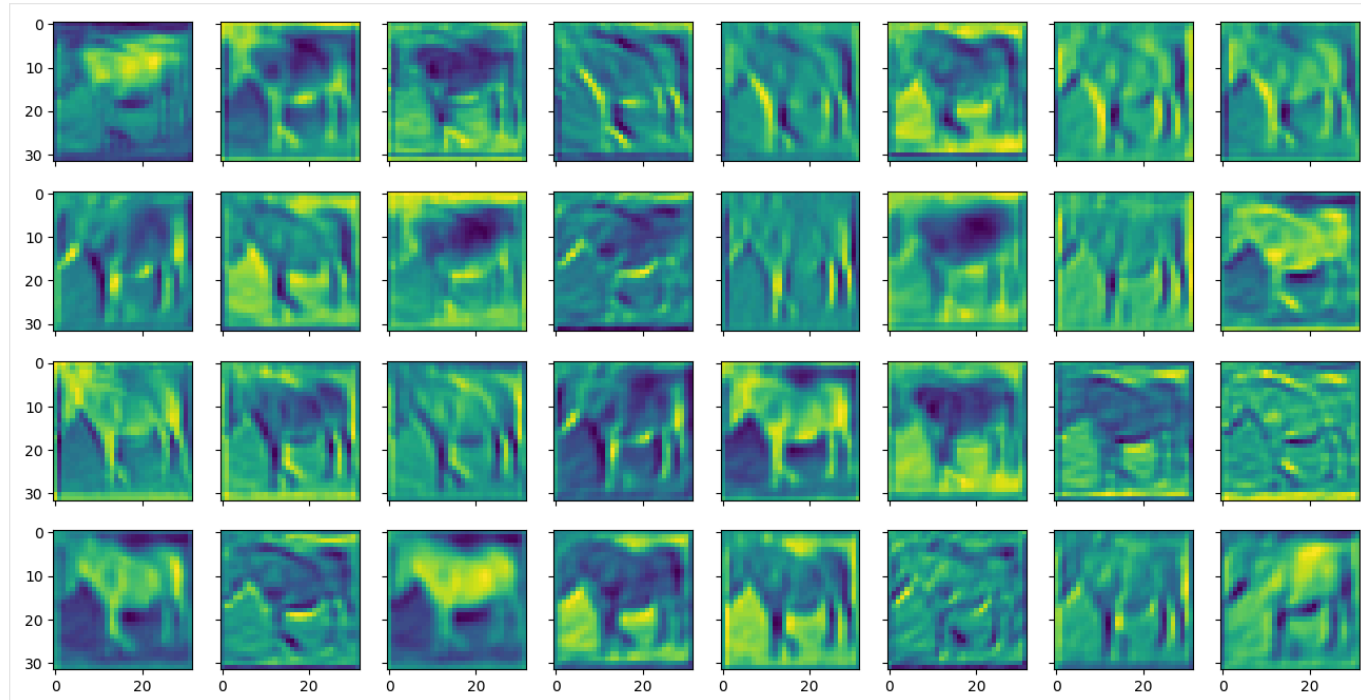


You can see that they are called feature maps because they are highlighting certain features from the input image. A feature is identified using a small window (in this case, over a 3x3 pixels filter). The input image has three color channels. Each channel has a different filter applied, and their results are combined for an output feature.

You can similarly display the feature map from the output of the second convolutional layer as follows:

```
1 X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
2
3 model.eval()
4 with torch.no_grad():
5     feature_maps = model.act1(model.conv1(X))
6     feature_maps = model.drop1(feature_maps)
7     feature_maps = model.conv2(feature_maps)
8
9 fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
10 for i in range(0, 32):
11     row, col = i//8, i%8
12     ax[row][col].imshow(feature_maps[0][i])
13 plt.show()
```

Which shows:



Compared to the output of the first convolutional layer, the feature maps from the second convolutional layer looks blurry and more abstract. But these are more useful for the model to identify the objects.

Putting everything together, the code below loads the saved model from the previous section and generate the feature maps:

```
1 import torch
2 import torch.nn as nn
3 import torchvision
4 import matplotlib.pyplot as plt
5
6 trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)
7
8 class CIFAR10Model(nn.Module):
9     def __init__(self):
10         super().__init__()
11         self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
12         self.act1 = nn.ReLU()
13         self.drop1 = nn.Dropout(0.3)
14
15         self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
16         self.act2 = nn.ReLU()
17         self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))
18
19         self.flat = nn.Flatten()
20
21         self.fc3 = nn.Linear(8192, 512)
22         self.act3 = nn.ReLU()
23         self.drop3 = nn.Dropout(0.5)
24
25         self.fc4 = nn.Linear(512, 10)
26
27     def forward(self, x):
28         # input 3x32x32, output 32x32x32
29         x = self.act1(self.conv1(x))
30         x = self.drop1(x)
31         # input 32x32x32, output 32x32x32
32         x = self.act2(self.conv2(x))
33         # input 32x32x32, output 32x16x16
34         x = self.pool2(x)
35         # input 32x16x16, output 8192
36         x = self.flat(x)
37         # input 8192, output 512
38         x = self.act3(self.fc3(x))
39         x = self.drop3(x)
40         # input 512, output 10
41         x = self.fc4(x)
42         return x
43
44 model = CIFAR10Model()
45 model.load_state_dict(torch.load("cifar10model.pth"))
46
47 plt.imshow(trainset.data[7])
48 plt.show()
49
50 X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
51 model.eval()
52 with torch.no_grad():
53     feature_maps = model.conv1(X)
```



```

54 fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
55 for i in range(0, 32):
56     row, col = i//8, i%8
57     ax[row][col].imshow(feature_maps[0][i])
58 plt.show()
59
60 with torch.no_grad():
61     feature_maps = model.act1(model.conv1(X))
62     feature_maps = model.drop1(feature_maps)
63     feature_maps = model.conv2(feature_maps)
64 fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
65 for i in range(0, 32):
66     row, col = i//8, i%8
67     ax[row][col].imshow(feature_maps[0][i])
68 plt.show()

```

## Further Readings

This section provides more resources on the topic if you are looking to go deeper.

### Posts

- [How Do Convolutional Layers Work in Deep Learning Neural Networks?](#)
- [Training a Classifier, from PyTorch tutorials](#)

### Books

- Chapter 9: Convolutional Networks, [Deep Learning](#), 2016.

### APIs

- [nn.Conv2d](#) layer in PyTorch

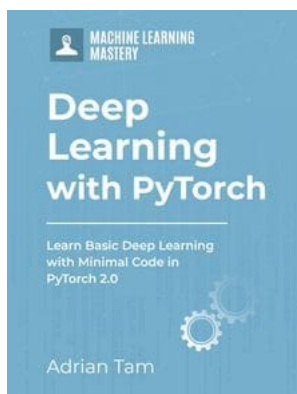
## Summary

In this post, you learned how to use convolutional neural network to handle image input and how to visualize the feature maps.

Specifically, you learned:

- The structure of a typical convolutional neural networks
- What is the effect of the filter size to a convolutional layer
- What is the effect of stacking convolutional layers in a network
- How to extract and visualize feature maps from a convolutional neural network

## Get Started on Deep Learning with PyTorch!



### Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:  
[Deep Learning with PyTorch](#)

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

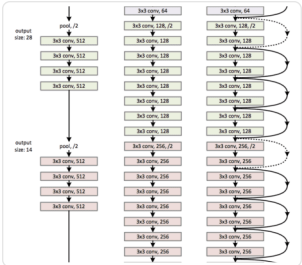
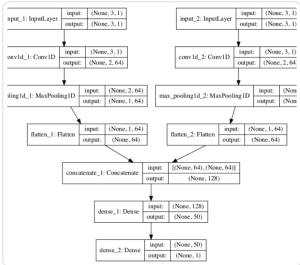
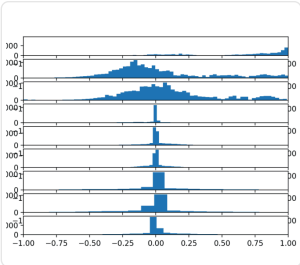
**Kick-start your deep learning journey with hands-on exercises**

[SEE WHAT'S INSIDE](#)

[Share](#) [Share](#)

Post

### More On This Topic



Understanding the Design of a Convolutional Neural Network

Building a Single Layer Neural Network in PyTorch



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.  
View all posts by Adrian Tam →

< Visualizing a PyTorch Model

Handwritten Digit Recognition with LeNet5 Model in PyTorch >

4 Responses to Building a Convolutional Neural Network in PyTorch



Prashanth Reddy Nimmakayala March 29, 2023 at 4:19 pm #

REPLY ↩

In the code section “An Example of Convolutional Neural Network” we find in the line #L13  
testloader = torch.utils.data.DataLoader( #here => trainset, batch\_size=batch\_size, shuffle=True)  
we should be having testset instead of trainset right?



James Carmichael March 30, 2023 at 7:12 am #

REPLY ↩

Hi Prashanth...Yes. Thank you for your feedback!



Oiomega March 1, 2024 at 2:44 pm #

REPLY ↩

You don't explain or mention the class at all. As a beginner it was hard to understand and follow the whole guide with a giant hole being present within the process.



James Carmichael March 2, 2024 at 1:08 pm #

REPLY ↩

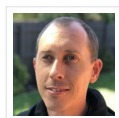


Thank you for your feedback!

## Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT



### Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

### Never miss a tutorial:



### Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Building a Regression Model in PyTorch

## Loving the Tutorials?

The [Deep Learning with PyTorch](#) EBook  
is where you'll find the ***Really Good*** stuff.

>> [SEE WHAT'S INSIDE](#)



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved